

Sentiment_Contagion_Analysis

September 18, 2026

```
[1]: import akshare as ak
import pandas as pd
from datetime import datetime

def get_hk_stock_data(symbol: str):
    """
    """

    try:
        #
        stock_data = ak.stock_hk_daily(symbol=symbol, adjust="qfq")

        if stock_data.empty:
            print(f" {symbol} ")
            return pd.DataFrame()

        print(f"{symbol}    shape: {stock_data.shape}")

        #    date    datetime
        if 'date' in stock_data.columns:
            stock_data['date'] = pd.to_datetime(stock_data['date'],
errors='coerce')
        else:
            print(f" {symbol}    'date' ")
            return pd.DataFrame()

        #
        keep_cols = ['date', 'open', 'high', 'low', 'close', 'volume']
        available_cols = [c for c in keep_cols if c in stock_data.columns]
        data = stock_data[available_cols].copy()

        #
        start_date = pd.to_datetime("2024-05-27")
        end_date = pd.to_datetime("2026-01-31")

        data = data[(data['date'] >= start_date) & (data['date'] <= end_date)]
```

```

    if data.empty:
        print(f" {symbol} {start_date.date()} ~ {end_date.date()} ")
        return pd.DataFrame()

    #
    data = data.set_index('date').sort_index()

    # --- reindex(full_index).ffill() ---
    # Ticker
    data['Ticker'] = symbol.zfill(5)

    print(f"{symbol} shape: {data.shape} ( )")
    return data

except Exception as e:
    print(f" {symbol} : {str(e)}")
    return pd.DataFrame()

def save_multiple_hk_stocks(tickers, filename):
    all_data = []
    for ticker in tickers:
        df = get_hk_stock_data(ticker)
        if not df.empty:
            all_data.append(df)
        print(f" {ticker}, : {len(df)}")

    if not all_data:
        print(" ")
        return

    final_df = pd.concat(all_data)

    # 'date'
    final_df = final_df.reset_index(names='date')

    #
    final_df['date'] = final_df['date'].dt.strftime('%Y/%m/%d')

    #
    final_df.to_csv(filename, encoding="utf-8-sig", index=False)
    print(f" : {filename}")
    print(f" : {len(final_df)}, : {final_df.columns.tolist()}")

tickers = ["00700", "09988", "01810"]
save_multiple_hk_stocks(tickers, "data/hk_stocks.csv")

```

```

00700    shape: (5472, 6)
00700    shape: (416, 6) ( )
      00700,    : 416
09988    shape: (1675, 6)
09988    shape: (416, 6) ( )
      09988,    : 416
01810    shape: (2018, 6)
01810    shape: (416, 6) ( )
      01810,    : 416
      : data/hk_stocks.csv
      : 1248,    : ['date', 'open', 'high', 'low', 'close', 'volume', 'Ticker']

```

```

[2]: import pandas as pd
import re
import os
OUTPUT_RAW_DIR = "data/raw"
INPUT_DIR = "data/weibo_hot_history"
# (00700.HK)
words_tencent = [
    #
    ' ',
    # /
    ' ', 'QQ', ' ', ' ', ' ', ' ',
    #
    ' ', ' ',
]
# (09988.HK)
words_alibaba = [
    #
    ' ', #
    ' ', ' ', ' ', ' ', ' ', ' ', #
    ' ', #
    '11', '618' # / /
]
# (01810.HK)
words_xiaomi = [
    #
    ' ',
    # /
    'SU7'
    #
    ' ',
]

def parse_md_file(filename):
    """ md """
    with open(filename, "r", encoding="utf-8") as f:

```

```

    text = f.read()

    #
    pattern = r'(\d{4}-\d{2}-\d{2})'          #   YYYY-MM-DD

    match = re.search(pattern, filename)
    if match:
        date = match.group(1)
    else:
        print(" ")

    #
    pattern = re.compile(r"(\d+)\.\s+\[(.*?)\]\(.*?)\s+`(.*)`\s+-\s+(\d+)")
    matches = pattern.findall(text)

    data = []
    for rank, keyword, category, hot_value in matches:
        hot_value_int = int(hot_value)
        #   hot_value  0
        if hot_value_int == 0:
            continue

        #
        keyword_lower = keyword.lower()
        category_lower = category.lower()

        is_tencent = any(word.lower() in keyword_lower or word.lower() in
↪category_lower
                        for word in words_tencent)

        is_alibaba = any(word.lower() in keyword_lower or word.lower() in
↪category_lower
                        for word in words_alibaba)

        is_xiaomi = any(word.lower() in keyword_lower or word.lower() in
↪category_lower
                        for word in words_xiaomi)

        data.append({
            "rank": int(rank),
            "keyword": keyword,
            "category": category,
            "hot_value": int(hot_value),
            "date": date,
            "is_tencent": is_tencent,
            "is_alibaba": is_alibaba,
            "is_xiaomi": is_xiaomi
        })

    return data

```

```

def parse_all_files(folder_path):
    """    md    """
    all_data = []
    for file in os.listdir(folder_path):
        if file.endswith(".md"):
            filepath = os.path.join(folder_path, file)
            # print(f"Parsing {filepath} ...")
            all_data.extend(parse_md_file(filepath))
    df = pd.DataFrame(all_data)
    return df

```

```

[3]: import pandas as pd
from transformers import pipeline
import torch
import os
from sklearn.model_selection import train_test_split

#
MODEL_NAME = "IDEA-CCNL/Erlangshen-Roberta-110M-Sentiment"
DEVICE = "mps" if torch.backends.mps.is_available() else "cpu"
print(f"Using device: {DEVICE}")

EWMA_SPAN = 7
OUTPUT_RAW_DIR = "data/raw"
OUTPUT_PROCESSED_DIR = "data/processed"
os.makedirs(OUTPUT_PROCESSED_DIR, exist_ok=True)

#
sentiment_pipeline = pipeline(
    "sentiment-analysis",
    model=MODEL_NAME,
    device=DEVICE,
    dtype=torch.float16 if DEVICE != "cpu" else None
)

def get_sentiment_score(text):
    """    [-1, 1]    """
    if pd.isna(text) or not text.strip():
        return 0.0
    try:
        result = sentiment_pipeline(text, truncation=True, max_length=512)[0]
        if '1' in result['label'] or 'positive' in result['label'].lower():
            return result['score']
        else:
            return -result['score']
    
```

```

except Exception as e:
    print(f"      : {text[:30]}... → {e}")
    return 0.0

#
# =====
#      "      "
THRESHOLD = 0.7
print("\n      ...")

train_output_path = os.path.join(OUTPUT_PROCESSED_DIR, "train_data.csv")
test_output_path = os.path.join(OUTPUT_PROCESSED_DIR, "test_data.csv")
#
if os.path.exists(train_output_path) and os.path.exists(test_output_path):
    print(f"      ")
else:
    # 1.
    raw_texts = pd.concat([
        df_tencent['keyword'],
        df_alibaba['keyword'],
        df_xiaomi['keyword']
    ]).reset_index(drop=True)

    # 2.      pipeline
    print("      ...")
    scores = [get_sentiment_score(text) for text in raw_texts]

    # 3.      (Pseudo-Labels)
    pseudo_labels = []
    valid_indices = [] #

    for i, score in enumerate(scores):
        if score > THRESHOLD:
            pseudo_labels.append(1)
            valid_indices.append(i)
        elif score < -THRESHOLD:
            pseudo_labels.append(0)
            valid_indices.append(i)

    # 4.      DataFrame
    df_for_split = pd.DataFrame({
        'text': raw_texts.iloc[valid_indices], #
        'label': pseudo_labels
    })

```

```

print(f"      {len(raw_texts)}          {len(df_for_split)}  ")

#5.      (80% , 20% )
train_df, test_df = train_test_split(
    df_for_split,
    test_size=0.2,
    random_state=42,      #
    stratify=pseudo_labels #
)
# 6.      CSV
train_df.to_csv(train_output_path, index=False, encoding="utf-8-sig")
print(f"      : {train_output_path} ({len(train_df)}  )")
test_df.to_csv(test_output_path, index=False, encoding="utf-8-sig")
print(f"      : {test_output_path} ({len(test_df)}  )")

print(f"      {len(test_df)}  ")
print(f"      : {test_output_path}")
print(f"      :\n{test_df.head()}")

```

Using device: mps

Device set to use mps

...

```

[4]: # --- Soft Label ---
import pandas as pd
import torch
import numpy as np
import os
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments,
    DataCollatorWithPadding,
    pipeline
)
from torch.utils.data import Dataset
from torch.nn import BCEWithLogitsLoss

# ===== 1. =====
MODEL_NAME = "IDEA-CCNL/Erlangshen-Roberta-110M-Sentiment"
MAX_LENGTH = 128
BATCH_SIZE = 32

```

```

EPOCHS = 2                                #
LEARNING_RATE = 2e-5                      #

DEVICE = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.
    is_available() else "cpu"
print(f"    : {DEVICE}")

# ===== 2. =====
print("    ...")
train_df = pd.read_csv('data/processed/train_data.csv')
test_df = pd.read_csv('data/processed/test_data.csv')
print(f"    : {len(train_df)} ")
print(f"    : {len(test_df)} ")

# ===== 3. =====
print("    ...")
sentiment_pipeline = pipeline(
    "sentiment-analysis",
    model=MODEL_NAME,
    device=0 if DEVICE == "cuda" else -1,
    truncation=True,
    max_length=512
)

def get_soft_label(text):
    if pd.isna(text) or not str(text).strip():
        return 0.5
    try:
        result = sentiment_pipeline(str(text))[0]
        score = result['score']
        if '1' in result['label'] or 'positive' in result['label'].lower():
            return score
        else:
            return 1 - score
    except:
        return 0.5

#
train_df['soft_label'] = train_df['text'].apply(get_soft_label)
print("    ")
print(train_df[['text', 'label', 'soft_label']].head())

# ===== 4. Dataset =====
class StockSentimentDataset(Dataset):
    def __init__(self, dataframe, tokenizer, max_length, is_train=True):
        self.tokenizer = tokenizer
        self.texts = dataframe['text'].tolist()

```

```

        self.max_length = max_length
        self.is_train = is_train
        if is_train:
            self.labels = dataframe['soft_label'].tolist()
        else:
            self.labels = dataframe['label'].tolist()

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = self.labels[idx]
        encoding = self.tokenizer(
            text,
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.float if self.is_train
↪else torch.long)
        }

# ===== 5. =====
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
train_dataset = StockSentimentDataset(train_df, tokenizer, MAX_LENGTH,
↪is_train=True)
test_dataset = StockSentimentDataset(test_df, tokenizer, MAX_LENGTH,
↪is_train=False)

print(f"      : {MODEL_NAME} ...")
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME,
↪num_labels=2)
model.to(DEVICE)

# ===== 6. =====
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='binary', zero_division=0
    )
    acc = accuracy_score(labels, preds)

```

```

    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall
    }

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

# ===== 7. Epoch 0 =====
print("\n    Epoch 0 ...")
temp_trainer = Trainer(
    model=model,
    args=TrainingArguments(output_dir='./tmp_pre', report_to="none",
    ↪per_device_eval_batch_size=BATCH_SIZE),
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
pre_results = temp_trainer.evaluate()

epoch0 = {
    'Epoch': 0,
    'Training Loss': None,
    'Validation Loss': pre_results.get('eval_loss'),
    'Accuracy': pre_results.get('eval_accuracy'),
    'F1': pre_results.get('eval_f1'),
    'Precision': pre_results.get('eval_precision'),
    'Recall': pre_results.get('eval_recall'),
}

# ===== 8. Soft Label Trainer =====
class SoftLabelTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits

        if labels.dtype == torch.float:
            #
            pos_logits = logits[:, 1]
            loss_fct = BCEWithLogitsLoss()
            loss = loss_fct(pos_logits, labels)
        else:
            #
            loss_fct = torch.nn.CrossEntropyLoss()
            loss = loss_fct(logits, labels)

```

```

        return (loss, outputs) if return_outputs else loss

# ===== 9. =====
training_args = TrainingArguments(
    output_dir='./results_soft',
    num_train_epochs=EPOCHS,
    per_device_train_batch_size=BATCH_SIZE,
    per_device_eval_batch_size=BATCH_SIZE,
    learning_rate=LEARNING_RATE,
    weight_decay=0.01,
    warmup_ratio=0.1,
    logging_dir='./logs',
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="eval_f1",
    greater_is_better=True,
    report_to="none",
    dataloader_pin_memory=False # MPS pin_memory
)

# ===== 10. =====
trainer = SoftLabelTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

print("\n Soft Label ...")
trainer.train()

# ===== 11. Epoch =====
print("\n" + "="*90)
print(" Epoch Epoch 0 ")
print("="*90)

history = trainer.state.log_history
results_list = [epoch0]

for i in range(1, EPOCHS + 1):
    #

```

```

        eval_log = next((log for log in history if log.get('epoch') == float(i) and
        ↪ 'eval_loss' in log), None)

        #     epoch     loss
        train_losses = [log['loss'] for log in history
                        if 'loss' in log and 'eval_loss' not in log and log.
        ↪ get('epoch', 0) <= float(i)]
        train_loss = train_losses[-1] if train_losses else None

        if eval_log:
            results_list.append({
                'Epoch': i,
                'Training Loss': train_loss,
                'Validation Loss': eval_log.get('eval_loss'),
                'Accuracy': eval_log.get('eval_accuracy'),
                'F1': eval_log.get('eval_f1'),
                'Precision': eval_log.get('eval_precision'),
                'Recall': eval_log.get('eval_recall'),
            })

df_results = pd.DataFrame(results_list)
print(df_results.to_string(index=False, float_format="%.6f"))
print("="*90)

# ===== 12. =====
print("\n    ...")
final_results = trainer.evaluate()
print("    :", final_results)

save_path = "./fine_tuned_sentiment_model_soft"
trainer.model.save_pretrained(save_path)
tokenizer.save_pretrained(save_path)
print(f"\n    : {save_path}")

```

: mps

...

: 1556

: 390

...

Device set to use cpu

		text	label	soft_label
0	10		0	0.213467
1	2024		1	0.862459
2	Turbo5 10		1	0.831094
3	618		1	0.884986

```

4          QQ          0      0.166351
      : IDEA-CCNL/Erlangshen-Roberta-110M-Sentiment ...

Epoch 0 ...

/var/folders/25/n2bknc210l370pvbdt5tcxm00000gn/T/ipykernel_2963/1758069359.py:12
2: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0
for `Trainer.__init__`. Use `processing_class` instead.
    temp_trainer = Trainer(
/opt/anaconda3/envs/myfinenv/lib/python3.10/site-
packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument
is set as true but not supported on MPS now, device pinned memory won't be used.
    super().__init__(loader)

<IPython.core.display.HTML object>

/var/folders/25/n2bknc210l370pvbdt5tcxm00000gn/T/ipykernel_2963/1758069359.py:18
0: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0
for `SoftLabelTrainer.__init__`. Use `processing_class` instead.
    trainer = SoftLabelTrainer(
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and
generation config. The model config and generation config were aligned
accordingly, being updated with the tokenizer's values. Updated tokens:
{'eos_token_id': None, 'bos_token_id': None, 'pad_token_id': 0}.

Soft Label ...

<IPython.core.display.HTML object>

=====
=====
Epoch      Epoch 0
=====
=====
Epoch  Training Loss  Validation Loss  Accuracy      F1  Precision  Recall
0         NaN          0.443382  0.869231  0.903955   0.983607  0.836237
1       0.343300        0.541284  0.874359  0.909091   0.972222  0.853659
2       0.333400        0.594596  0.864103  0.900563   0.975610  0.836237
=====
=====

...

<IPython.core.display.HTML object>

      : {'eval_loss': 0.541284441947937, 'eval_accuracy': 0.8743589743589744,
'eval_f1': 0.9090909090909091, 'eval_precision': 0.9722222222222222,
'eval_recall': 0.8536585365853658, 'eval_runtime': 4.4479,
'eval_samples_per_second': 87.681, 'eval_steps_per_second': 2.923, 'epoch': 2.0}

```

```
: ./fine_tuned_sentiment_model_soft
```

```
[6]: import pandas as pd
from transformers import pipeline
import torch
import os

tuned_sentiment_pipeline = pipeline(
    "sentiment-analysis",
    model="./fine_tuned_sentiment_model_soft", #
    device=DEVICE,
    dtype=torch.float16 if DEVICE != "cpu" else None
)

def get_tuned_sentiment_score(text):
    """
    [-1, 1]
    """
    if pd.isna(text) or not isinstance(text, str) or not text.strip():
        return 0.0

    try:
        result = tuned_sentiment_pipeline(text, truncation=True,
        ↪max_length=MAX_LENGTH)[0]
        label = result['label']
        score = result['score']

        # LABEL_0/LABEL_1 -1 1
        # LABEL_1 LABEL_0
        final_score = score if label == 'Positive' else -score

        return round(final_score, 4)

    except Exception as e:
        print(f" : {str(text)[:30]}... → {e}")
        return 0.0

def compute_daily_sentiment(df_stock, stock_name):
    if df_stock.empty:
        print(f"{stock_name} ")
        return pd.DataFrame(columns=[
            'pos_sentiment', 'neg_sentiment', 'net_sentiment',
            'topic_count', 'pos_heat', 'neg_heat', 'total_heat'
        ])
    ])
```

```

df_stock = df_stock.copy()
df_stock['date'] = pd.to_datetime(df_stock['date'], errors='coerce')
df_stock = df_stock.dropna(subset=['date']).sort_values('date')

grouped = df_stock.groupby('date')
results = []

for date, group in grouped:
    group = group.copy()
    group['score'] = group['keyword'].apply(get_tuned_sentiment_score)

    total_heat = group['hot_value'].sum()
    if total_heat <= 0:
        results.append({
            'date': date,
            'pos_sentiment': 0.0,
            'neg_sentiment': 0.0,
            'net_sentiment': 0.0,
            'topic_count': len(group),
            'pos_heat': 0.0,
            'neg_heat': 0.0,
            'total_heat': 0.0
        })
        continue

    #
    pos_mask = group['score'] > 0
    pos_heat = group.loc[pos_mask, 'hot_value'].sum()
    if pos_heat > 0:
        pos_sentiment = (group.loc[pos_mask, 'score'] * group.loc[pos_mask,
↪ 'hot_value']).sum() / pos_heat
    else:
        pos_sentiment = 0.0

    #
    neg_mask = group['score'] < 0
    neg_heat = group.loc[neg_mask, 'hot_value'].sum()
    if neg_heat > 0:
        neg_sentiment = (group.loc[neg_mask, 'score'].abs() * group.
↪ loc[neg_mask, 'hot_value']).sum() / neg_heat
    else:
        neg_sentiment = 0.0

    #
    net_sentiment = (group['score'] * group['hot_value']).sum() / total_heat

    results.append({

```

```

        'date': date,
        'pos_sentiment': round(pos_sentiment, 4),
        'neg_sentiment': round(neg_sentiment, 4),
        'net_sentiment': round(net_sentiment, 4),
        'topic_count': len(group),
        'pos_heat': pos_heat,
        'neg_heat': neg_heat,
        'total_heat': total_heat
    })

df_daily = pd.DataFrame(results).set_index('date').sort_index()

# EWMA
for col in ['pos_sentiment', 'neg_sentiment', 'net_sentiment']:
    df_daily[f'{col}_smooth'] = df_daily[col].ewm(span=EWMA_SPAN,
↪adjust=False).mean().round(4)

print(f"{stock_name}: {len(df_daily)} // ")
return df_daily

```

Device set to use mps

```

[8]: #
if __name__ == "__main__":
    df = parse_all_files(INPUT_DIR)

    # True
    df_tencent = df[df["is_tencent"]].drop(columns=["is_tencent", "is_alibaba",
↪"is_xiaomi"])
    df_alibaba = df[df["is_alibaba"]].drop(columns=["is_tencent", "is_alibaba",
↪"is_xiaomi"])
    df_xiaomi = df[df["is_xiaomi"]].drop(columns=["is_tencent", "is_alibaba",
↪"is_xiaomi"])

    os.makedirs("data", exist_ok=True)

    df_tencent.to_csv(os.path.join(OUTPUT_RAW_DIR, "tencent.csv"), index=False,
↪encoding="utf-8-sig")
    df_alibaba.to_csv(os.path.join(OUTPUT_RAW_DIR, "alibaba.csv"), index=False,
↪encoding="utf-8-sig")
    df_xiaomi.to_csv(os.path.join(OUTPUT_RAW_DIR, "xiaomi.csv"), index=False,
↪encoding="utf-8-sig")

    print(" tencent.csv, alibaba.csv, xiaomi.csv, path = " + OUTPUT_RAW_DIR)

#

```

```

df_tencent = pd.read_csv(os.path.join(OUTPUT_RAW_DIR, "tencent.csv"),
encoding="utf-8-sig")
df_alibaba = pd.read_csv(os.path.join(OUTPUT_RAW_DIR, "alibaba.csv"),
encoding="utf-8-sig")
df_xiaomi = pd.read_csv(os.path.join(OUTPUT_RAW_DIR, "xiaomi.csv"),
encoding="utf-8-sig")

#
sent_tencent = compute_daily_sentiment(df_tencent, " ")
sent_alibaba = compute_daily_sentiment(df_alibaba, " ")
sent_xiaomi = compute_daily_sentiment(df_xiaomi, " ")

#      4
for df in [sent_tencent, sent_alibaba, sent_xiaomi]:
    for col in ['pos_sentiment', 'neg_sentiment', 'net_sentiment',
                'pos_sentiment_smooth', 'neg_sentiment_smooth',
net_sentiment_smooth']:
        if col in df.columns:
            df[col] = df[col].round(4)

# ===== daily_sentiment.csv =====
# pos_tencent / neg_tencent / net_tencent

merged = pd.concat([
    #
    sent_tencent['pos_sentiment_smooth'].rename('pos_tencent'),
    sent_tencent['neg_sentiment_smooth'].rename('neg_tencent'),
    sent_tencent['net_sentiment_smooth'].rename('net_tencent'),

    #
    sent_alibaba['pos_sentiment_smooth'].rename('pos_alibaba'),
    sent_alibaba['neg_sentiment_smooth'].rename('neg_alibaba'),
    sent_alibaba['net_sentiment_smooth'].rename('net_alibaba'),

    #
    sent_xiaomi['pos_sentiment_smooth'].rename('pos_xiaomi'),
    sent_xiaomi['neg_sentiment_smooth'].rename('neg_xiaomi'),
    sent_xiaomi['net_sentiment_smooth'].rename('net_xiaomi'),
], axis=1)

#
merged.to_csv(
    os.path.join(OUTPUT_PROCESSED_DIR, "daily_sentiment.csv"),
    encoding="utf-8-sig",
    na_rep='NaN',
    index=True
)

```

```

#
sent_tencent.to_csv(os.path.join(OUTPUT_PROCESSED_DIR, "sentiment_tencent.
↪CSV"),
                    encoding="utf-8-sig", na_rep='NaN', index=True)
sent_alibaba.to_csv(os.path.join(OUTPUT_PROCESSED_DIR, "sentiment_alibaba.
↪CSV"),
                    encoding="utf-8-sig", na_rep='NaN', index=True)
sent_xiaomi.to_csv(os.path.join(OUTPUT_PROCESSED_DIR, "sentiment_xiaomi.
↪CSV"),
                   encoding="utf-8-sig", na_rep='NaN', index=True)

print("\n      CSV  ")
print(f" {OUTPUT_PROCESSED_DIR}/daily_sentiment.csv      pos/neg/net ")
print(f" {OUTPUT_PROCESSED_DIR}/sentiment_tencent.csv")
print(f" {OUTPUT_PROCESSED_DIR}/sentiment_alibaba.csv")
print(f" {OUTPUT_PROCESSED_DIR}/sentiment_xiaomi.csv")
print(" pos_tencent, neg_tencent, net_tencent ...")

#      CSV      CSV
merged.to_csv(os.path.join(OUTPUT_PROCESSED_DIR, "daily_sentiment.csv"),
              encoding="utf-8-sig",
              na_rep='NaN',
              index=True) #      date

#      CSV
sent_tencent.to_csv(os.path.join(OUTPUT_PROCESSED_DIR, "sentiment_tencent.
↪CSV"),
                    encoding="utf-8-sig",
                    na_rep='NaN',
                    index=True)

sent_alibaba.to_csv(os.path.join(OUTPUT_PROCESSED_DIR, "sentiment_alibaba.
↪CSV"),
                    encoding="utf-8-sig",
                    na_rep='NaN',
                    index=True)

sent_xiaomi.to_csv(os.path.join(OUTPUT_PROCESSED_DIR, "sentiment_xiaomi.
↪CSV"),
                   encoding="utf-8-sig",
                   na_rep='NaN',
                   index=True)

print("\n      CSV  ")
print(f" {OUTPUT_PROCESSED_DIR}/daily_sentiment.csv      (      )")

```

```

print(f" {OUTPUT_PROCESSED_DIR}/sentiment_tencent.csv ( )")
print(f" {OUTPUT_PROCESSED_DIR}/sentiment_alibaba.csv ( )")
print(f" {OUTPUT_PROCESSED_DIR}/sentiment_xiaomi.csv ( )")
print(" NaN ")

```

```

tencent.csv, alibaba.csv, xiaomi.csv, path = data/raw
: 481 //
: 245 //
: 334 //

```

```

CSV
data/processed/daily_sentiment.csv      pos/neg/net
data/processed/sentiment_tencent.csv
data/processed/sentiment_alibaba.csv
data/processed/sentiment_xiaomi.csv
pos_tencent, neg_tencent, net_tencent ...

```

```

CSV
data/processed/daily_sentiment.csv      ( )
data/processed/sentiment_tencent.csv    ( )
data/processed/sentiment_alibaba.csv    ( )
data/processed/sentiment_xiaomi.csv     ( )
NaN

```

```

[9]: import pandas as pd
import numpy as np
from scipy.stats import pearsonr, spearmanr
import matplotlib.pyplot as plt
import seaborn as sns
import os

print("=== - / / ===")

#
STOCK_FILE = "data/hk_stocks.csv"
SENTIMENT_FILE = os.path.join("data/processed", "daily_sentiment.csv")

# 1.
df_price = pd.read_csv(STOCK_FILE, parse_dates=['date'], date_format='%Y/%m/%d')
df_price['Ticker'] = df_price['Ticker'].astype(str).str.zfill(5)

# 2.
df_sent = pd.read_csv(SENTIMENT_FILE, parse_dates=['date'], index_col='date')
df_sent.index = pd.to_datetime(df_sent.index)

#
# pos_tencent / neg_tencent / net_tencent

```

```

# pos_alibaba / neg_alibaba / net_alibaba
# pos_xiaomi / neg_xiaomi / net_xiaomi
tickers = {
    '00700': (' ', 'tencent'),
    '09988': (' ', 'alibaba'),
    '01810': (' ', 'xiaomi')
}

#
sentiment_types = {
    'pos': ' ',
    'neg': ' ',
    'net': ' '
}

results = [] #

for ticker, (name, stock_key) in tickers.items():
    # ---
    df_p = df_price[df_price['Ticker'] == ticker].copy().set_index('date').
    ↪sort_index()
    df_p['ret'] = df_p['close'].pct_change() #

    print(f"\n{'='*60}")
    print(f" {name} {ticker} ")
    print(f"{'='*60}")

    for sent_key, sent_name in sentiment_types.items():
        sent_col = f"{sent_key}_{stock_key}" # pos_tencent

        if sent_col not in df_sent.columns:
            print(f" {sent_col} {sent_name}")
            continue

        #
        df_m = df_sent[[sent_col]].rename(columns={sent_col: 'sentiment'}).join(
            df_p[['ret']],
            how='right'
        )

        # ffill
        # limit=3 3
        df_m['sentiment'] = df_m['sentiment'].ffill(limit=3)

        #
        df_a = df_m.dropna(subset=['ret', 'sentiment']).copy()

```

```

if len(df_a) < 30:
    print(f" {sent_name} {len(df_a)} ")
    continue

#
r_p, p_p = pearsonr(df_a['sentiment'], df_a['ret'])
r_s, p_s = spearmanr(df_a['sentiment'], df_a['ret'])

print(f"\n {sent_name} vs ")
print(f" Pearson : {r_p:.4f} (p={p_p:.4f})")
print(f" Spearman: {r_s:.4f} (p={p_s:.4f})")

#
print(" ")
for lag in range(0, 6):
    temp = df_a.copy()
    temp['sent_lag'] = temp['sentiment'].shift(lag)
    temp = temp.dropna()

    if len(temp) > 20:
        r, p_value = pearsonr(temp['sent_lag'], temp['ret'])

        if p_value < 0.01:
            sig = "***"
        elif p_value < 0.05:
            sig = "**"
        elif p_value < 0.1:
            sig = "*"
        else:
            sig = ""

        print(f" {lag} → r: {r:.4f} (p={p_value:.4f}) {sig}")

#
df_a = df_a.copy()
df_a['stock'] = name
df_a['sentiment_type'] = sent_name
results.append(df_a)

print("\n=== ===")
print(f" {len(results)} × ")

```

=== - / / ===

=====

00700

=====

vs
Pearson : -0.0166 (p=0.7378)
Spearman: -0.0272 (p=0.5822)

0 → r: -0.0166 (p=0.7378)
1 → r: -0.0062 (p=0.8998)
2 → r: 0.0079 (p=0.8729)
3 → r: 0.0127 (p=0.7985)
4 → r: 0.0133 (p=0.7900)
5 → r: -0.0035 (p=0.9434)

vs
Pearson : -0.0472 (p=0.3409)
Spearman: -0.0417 (p=0.3994)

0 → r: -0.0472 (p=0.3409)
1 → r: -0.0510 (p=0.3039)
2 → r: -0.0331 (p=0.5048)
3 → r: 0.0236 (p=0.6343)
4 → r: 0.0262 (p=0.5982)
5 → r: 0.0498 (p=0.3171)

vs
Pearson : -0.0064 (p=0.8975)
Spearman: -0.0222 (p=0.6540)

0 → r: -0.0064 (p=0.8975)
1 → r: 0.0009 (p=0.9857)
2 → r: 0.0052 (p=0.9168)
3 → r: -0.0295 (p=0.5522)
4 → r: -0.0343 (p=0.4904)
5 → r: -0.0543 (p=0.2753)

=====

09988

=====

vs
Pearson : -0.0551 (p=0.3114)
Spearman: -0.0764 (p=0.1601)

0 → r: -0.0551 (p=0.3114)
1 → r: -0.0970 (p=0.0746) *
2 → r: -0.0687 (p=0.2077)
3 → r: -0.0432 (p=0.4298)
4 → r: -0.0542 (p=0.3219)
5 → r: -0.0854 (p=0.1188)

vs
Pearson : 0.0260 (p=0.6328)
Spearman: 0.0491 (p=0.3665)

0 → r: 0.0260 (p=0.6328)
1 → r: 0.0603 (p=0.2681)
2 → r: 0.0430 (p=0.4309)
3 → r: 0.0263 (p=0.6298)
4 → r: 0.0007 (p=0.9896)
5 → r: 0.0214 (p=0.6967)

vs
Pearson : -0.0710 (p=0.1913)
Spearman: -0.0951 (p=0.0799)

0 → r: -0.0710 (p=0.1913)
1 → r: -0.1029 (p=0.0584) *
2 → r: -0.0748 (p=0.1702)
3 → r: -0.0480 (p=0.3793)
4 → r: -0.0415 (p=0.4482)
5 → r: -0.0636 (p=0.2460)

=====

01810

=====

vs
Pearson : 0.0139 (p=0.7854)
Spearman: -0.0024 (p=0.9629)

0 → r: 0.0139 (p=0.7854)
1 → r: -0.0161 (p=0.7530)
2 → r: -0.0318 (p=0.5343)
3 → r: -0.0392 (p=0.4434)
4 → r: -0.0318 (p=0.5353)
5 → r: -0.0397 (p=0.4393)

vs
Pearson : -0.0667 (p=0.1902)
Spearman: -0.0365 (p=0.4746)

0 → r: -0.0667 (p=0.1902)
1 → r: -0.0498 (p=0.3294)
2 → r: -0.0144 (p=0.7777)
3 → r: -0.0301 (p=0.5571)
4 → r: -0.0543 (p=0.2888)
5 → r: -0.0923 (p=0.0717) *

```

vs
Pearson : 0.0400 (p=0.4326)
Spearman: 0.0095 (p=0.8523)

0  → r: 0.0400 (p=0.4326)
1  → r: 0.0306 (p=0.5493)
2  → r: 0.0065 (p=0.8995)
3  → r: -0.0040 (p=0.9381)
4  → r: -0.0174 (p=0.7336)
5  → r: 0.0039 (p=0.9394)

===      ===
9        x

```

```

[12]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr
import os

#
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

os.makedirs("data/images", exist_ok=True)

print("===      ===")

if 'results' not in globals() or len(results) == 0:
    raise ValueError("      results      ")

#
stock_map = {
    ' ': 'Tencent',
    ' ': 'Alibaba',
    ' ': 'Xiaomi'
}

#
colors = {
    'Tencent': '#1f77b4',
    'Alibaba': '#ff7f0e',
    'Xiaomi': '#2ca02c'
}

sentiment_colors = {

```

```

        ' ': '#2ca02c',    #
        ' ': '#d62728',    #
        ' ': '#1f77b4'      #
    }

# =====
# 1.
# =====
print("1.      ")

lag_data = []

for df in results:
    stock_cn = df['stock'].iloc[0]
    stock_en = stock_map.get(stock_cn, stock_cn)
    sent_type = df['sentiment_type'].iloc[0]

    best_abs_r = -np.inf
    best_lag = 0
    best_r = 0

    for lag in range(0, 6):
        temp = df.copy()
        temp['sent_lag'] = temp['sentiment'].shift(lag)
        temp = temp.dropna()
        if len(temp) > 20:
            r, _ = pearsonr(temp['sent_lag'], temp['ret'])
            if abs(r) > best_abs_r:
                best_abs_r = abs(r)
                best_r = r
                best_lag = lag

    lag_data.append({
        'Stock': stock_en,
        'Sentiment': sent_type,
        'Best Lag (days)': best_lag,
        'Max Pearson': round(best_r, 4)
    })

df_lag = pd.DataFrame(lag_data)
print(df_lag)

plt.figure(figsize=(12, 6))
sns.barplot(
    data=df_lag,
    x='Stock',
    y='Max Pearson',

```

```

        hue='Sentiment',
        palette=sentiment_colors
    )
    plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
    plt.title('Sentiment Leading Effect: Best Lag Correlation with Daily_
        ↪Return\n(Positive / Negative / Net)')
    plt.ylabel('Max Pearson Correlation (selected by |r|)')
    plt.xlabel('Stock')
    plt.legend(title='Sentiment Type')
    plt.grid(True, alpha=0.3, axis='y')
    plt.tight_layout()
    plt.savefig('data/images/lag_correlation_pos_neg_net.png', dpi=300,
        ↪bbox_inches='tight')
    plt.show()

# =====
# 2.    +    3    x    3    =    9
# =====
print("\n2.    ")

fig, axes = plt.subplots(3, 3, figsize=(16, 14))
axes = axes.flatten()

idx = 0
for df in results:
    stock_cn = df['stock'].iloc[0]
    stock_en = stock_map.get(stock_cn, stock_cn)
    sent_type = df['sentiment_type'].iloc[0]
    ax = axes[idx]

    sns.regplot(
        data=df,
        x='sentiment',
        y='ret',
        scatter_kws={'alpha': 0.5, 's': 25, 'color': sentiment_colors.
            ↪get(sent_type, '#9467bd')},
        line_kws={'color': 'black', 'linewidth': 1.5},
        ax=ax
    )
    ax.set_title(f'{stock_en} - {sent_type}', fontsize=11)
    ax.set_xlabel('Sentiment Score')
    ax.set_ylabel('Daily Return')
    ax.grid(True, alpha=0.3)
    ax.axhline(0, color='gray', linewidth=0.8, linestyle='--')
    ax.axvline(0, color='gray', linewidth=0.8, linestyle='--')
    idx += 1

```

```

plt.suptitle('Sentiment vs Daily Return (Positive / Negative / Net)',
             ↪fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('data/images/scatter_pos_neg_net.png', dpi=300, bbox_inches='tight')
plt.show()

# =====
# 3.    +
# =====
print("\n3.    ")

#
df_price = pd.read_csv("data/hk_stocks.csv", parse_dates=['date'],
                      ↪date_format='%Y/%m/%d')
df_price['Ticker'] = df_price['Ticker'].astype(str).str.zfill(5)

ticker_map = {'Tencent': '00700', 'Alibaba': '09988', 'Xiaomi': '01810'}
sent_file_map = {
    'Tencent': 'sentiment_tencent.csv',
    'Alibaba': 'sentiment_alibaba.csv',
    'Xiaomi': 'sentiment_xiaomi.csv'
}

for stock_en, ticker in ticker_map.items():
    #
    df_p = df_price[df_price['Ticker'] == ticker].set_index('date').sort_index()

    #
    sent_path = os.path.join("data/processed", sent_file_map[stock_en])
    if not os.path.exists(sent_path):
        print(f"    {stock_en}    {sent_path}")
        continue

    df_s = pd.read_csv(sent_path, parse_dates=['date'], index_col='date')
    df_s.index = pd.to_datetime(df_s.index)

    #
    df_plot = df_p[['close']].join(
        df_s[['pos_sentiment_smooth', 'neg_sentiment_smooth',
        ↪'net_sentiment_smooth']],
        how='inner'
    ).dropna()

    fig, ax1 = plt.subplots(figsize=(14, 6))

    #

```

```

    ax1.plot(df_plot.index, df_plot['close'], color=colors[stock_en],
    ↪linewidth=1.8, label=f'{stock_en} Close')
    ax1.set_ylabel('Stock Price (HKD)', color=colors[stock_en])
    ax1.tick_params(axis='y', labelcolor=colors[stock_en])

    #
    ax2 = ax1.twinx()
    ax2.plot(df_plot.index, df_plot['pos_sentiment_smooth'], color='#2ca02c',
    ↪alpha=0.8, label='Positive (EWMA)')
    ax2.plot(df_plot.index, df_plot['neg_sentiment_smooth'], color='#d62728',
    ↪alpha=0.8, label='Negative (EWMA)')
    ax2.plot(df_plot.index, df_plot['net_sentiment_smooth'], color='#1f77b4',
    ↪alpha=0.9, linewidth=1.5, label='Net (EWMA)')
    ax2.set_ylabel('Sentiment Score', color='gray')
    ax2.tick_params(axis='y', labelcolor='gray')
    ax2.axhline(0, color='gray', linewidth=0.8, linestyle='--')

    plt.title(f'{stock_en} Stock Price vs Positive / Negative / Net Sentiment')
    #
    lines1, labels1 = ax1.get_legend_handles_labels()
    lines2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left', fontsize=9)

    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(f'data/images/{stock_en.lower()}_price_sentiment_pos_neg_net.
    ↪png', dpi=300, bbox_inches='tight')
    plt.show()

print("\n===      data/images/ ===")
print("    ")
print(" - lag_correlation_pos_neg_net.png")
print(" - scatter_pos_neg_net.png")
print(" - tencent_price_sentiment_pos_neg_net.png")
print(" - alibaba_price_sentiment_pos_neg_net.png")
print(" - xiaomi_price_sentiment_pos_neg_net.png")

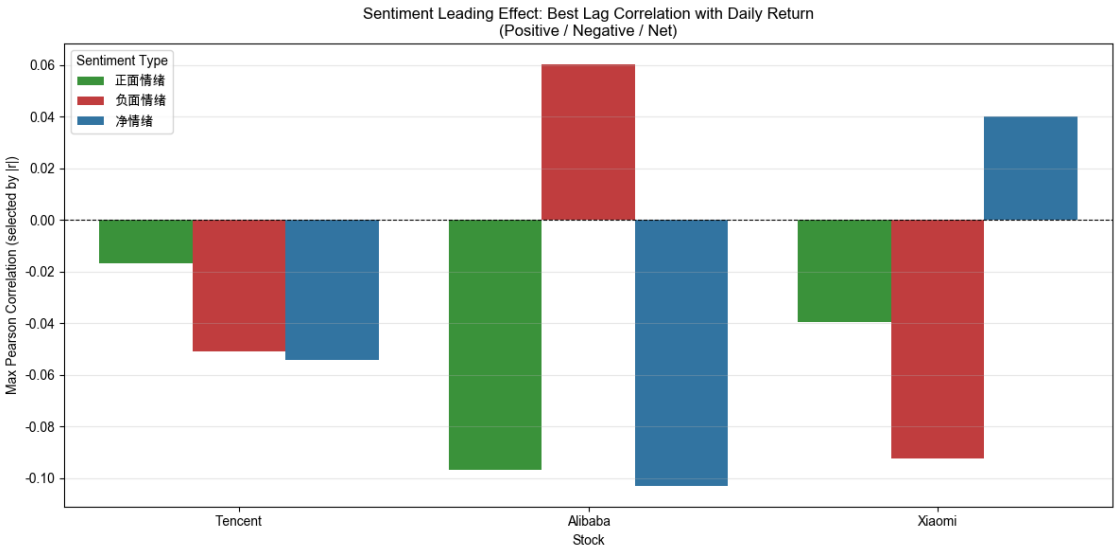
```

```

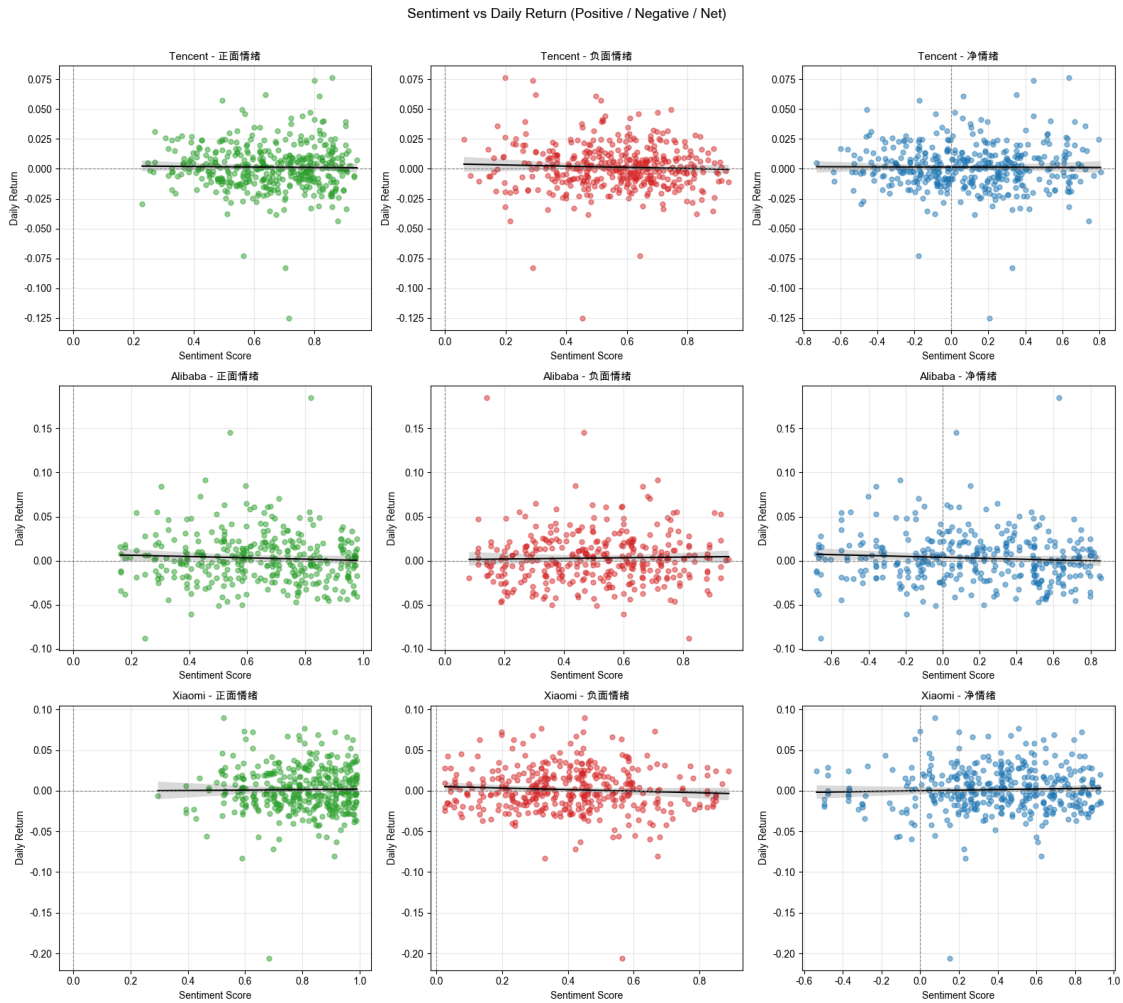
===      ===
1.
    Stock Sentiment  Best Lag (days)  Max Pearson
0  Tencent          0                -0.0166
1  Tencent          1                -0.0510
2  Tencent          5                -0.0543
3  Alibaba          1                -0.0970
4  Alibaba          1                 0.0603
5  Alibaba          1                -0.1029
6  Xiaomi           5                -0.0397

```

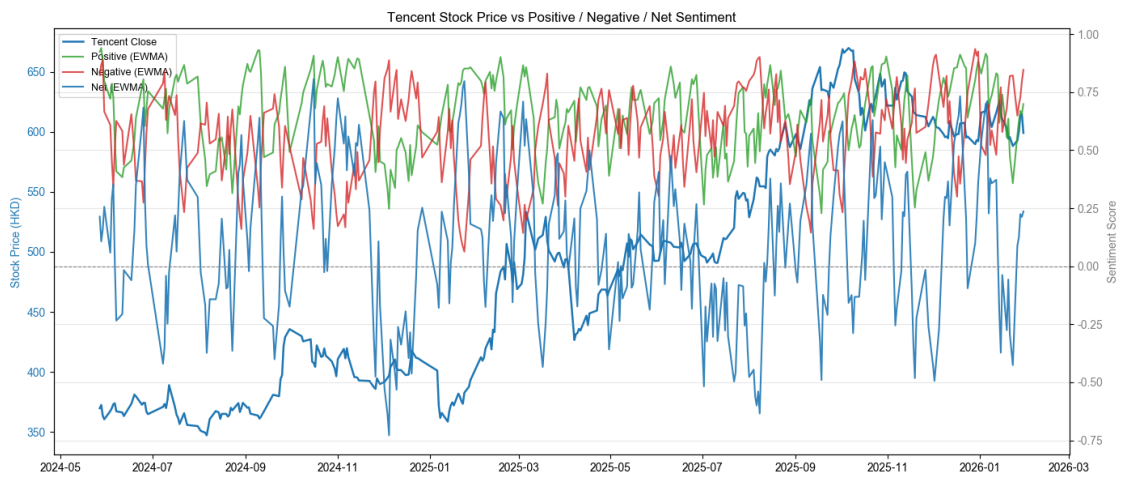
7	Xiaomi	5	-0.0923
8	Xiaomi	0	0.0400

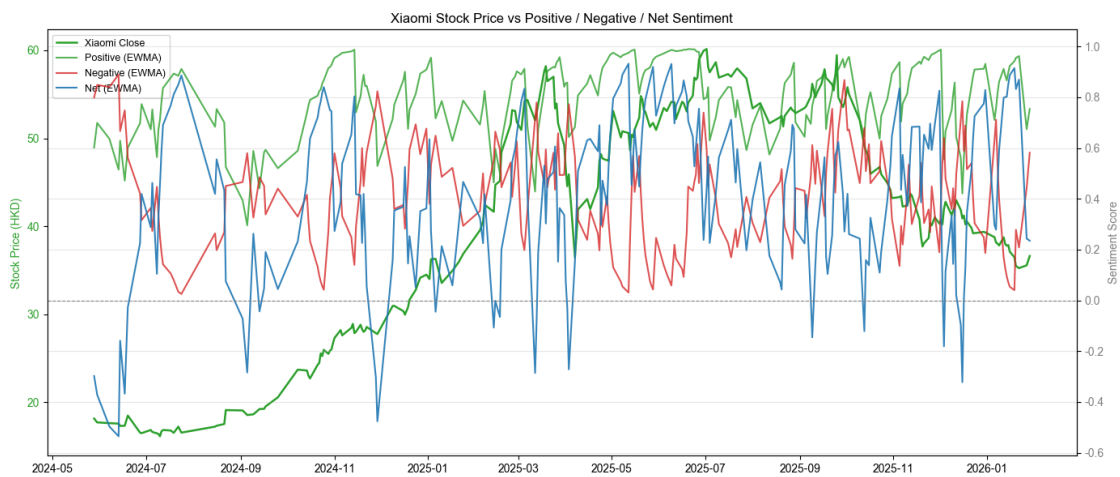
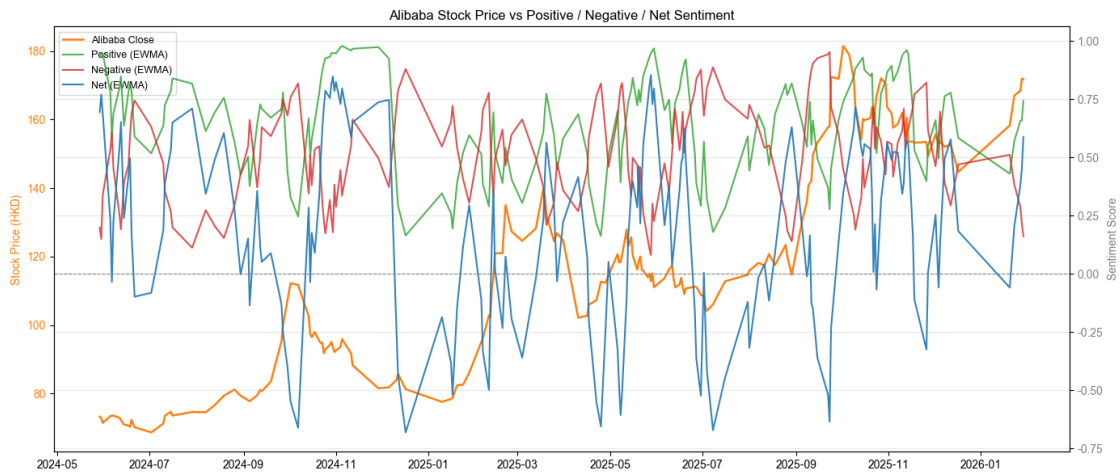


2.



3.





=== data/images/ ===

- lag_correlation_pos_neg_net.png
- scatter_pos_neg_net.png
- tencent_price_sentiment_pos_neg_net.png
- alibaba_price_sentiment_pos_neg_net.png
- xiaomi_price_sentiment_pos_neg_net.png

[]: